Reviewer Pack — Specht Character Support Bounds Monotone Permanental Formula…

Entry b5cefcac0023 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 09:45:18 UTC

Specht Character Support Bounds Monotone Permanental Formula Size

Entry ID: `b5cefcac0023` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 09:45:18 UTC

1. Conjecture

Field A (mathematical branch): Specht modules and Murnaghan-Nakayama character theory of the symmetric group S_n **Field B** (complexity object): Monotone arithmetic formula size for multilinear permanental polynomials (GCT det-vs-perm monotone setting)

Statement:

> For a multilinear permanental polynomial f = Σ_{σ∈S_n} c_f(σ) Π_i x_{i,σ(i)} in n×n matrix variables with c_f ≥ 0, define Specht coefficients α_λ(f) := (1/n!) Σ_σ c_f(σ) χ_λ(σ) for each partition λ ⊢ n, where χ_λ is the irreducible character (computed via Murnaghan-Nakayama), and the effective Specht support |supp_eff(f)| := (Σ_λ (dim V_λ · α_λ)^2)2 / Σ_λ (dim V_λ · α_λ)^4 (inverse participation ratio). Conjecture: every monotone permanental arithmetic formula F of size s (binary tree of ⊕ gates and ⊗ gates that multiply polynomials in disjoint row- AND column-subsets, with nonnegative leaf weights) computing f satisfies |supp_eff(f)| ≤ s; consequently any 'pure' polynomial like perm_n (|supp_eff|=1, character (n)) or its sign-twin det_n (character (1^n)) requires monotone permanental size ≥ 1 trivially, but every monotone formula F that produces a polynomial f with |supp_eff(f)|=k must have size ≥ k. Falsifier: a single monotone permanental formula F with size s < |supp_eff(f_F)|.

Rationale (proposer's reasoning):

> Mulmuley-Sohoni view perm vs det through representation theory of GL-orbit closures; we drop to the more elementary diagonal S_n action where Murnaghan-Nakayama gives O(n! · p(n))-time exact computation of α_λ, bypassing both the natural-proofs barrier (the invariant is relational/categorical — a participation ratio across irreducible isotypics, not a single truth-table number) and algebrization (the construction is not a polynomial-ring identity in the boolean variables but a representation-theoretic projection on S_n cosets). The bound |supp_eff(f)| ≤ s reflects the intuition that each monotone gate can only construct one new 'pure character direction', so a formula must have at least as many gates as the character spread of its output — a quantitative refinement of the Jerrum-Snir style monotone obstruction.

Taxonomy category: `ACC_LB_via_WILLIAMS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `837377149fb9254f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across n∈{4,5,6,7}, s∈{1,2,4,8,16,32}, 30 seeds each (720 trials), measure r := |supp_eff(f_F)|/s per trial. Conjecture is SUPPORTED iff ≥80% of trials satisfy r ≤ 1 AND mean(r) ≤ 1 AND max(r) ≤ 1.05; FALSIFIED if any single seed yields r > 1.05 (i.e. |supp_eff| > 1.05·s).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `monotone arithmetic formula permanent lower bound symmetric group representation` - `Specht module support multilinear permanental polynomial geometric complexity theory` - `Murnaghan-Nakayama character monotone circuit complexity permanent determinant`

Top relevant hits considered: - [s2:3232ed274e19727bb02719b0038eeab0dad190c7] Stability in Topology, Arithmetic, and Representation Theory 2022 - [s2:a386ca523f19c66f4e550fdeff35ca56408afaa6] STACS 2006, 23rd Annual Symposium on Theoretical Aspects of Computer Science, Marseille, France, February 23-25, 2006, P

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def factorial(n):
    if n == 0 or n == 1:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def binomial_coefficient(n, k):
    return factorial(n) // (factorial(k) * factorial(n - k))
```

```
def murnaghan_nakayama(lam, mu, beta):
    if len(mu) > len(lam):
        return 0
    sign = 1
    for i in range(len(mu)):
        sign *= (-1) ** (lam[i] - mu[i])
    result = binomial_coefficient(lam[0], mu[0])
    for i in range(1, len(lam)):
        if lam[i] < mu[i]:
            return 0
        result *= binomial_coefficient(lam[i] - mu[i], mu[i]) // binomial_coefficient(lam[i], mu[i
    return sign * result

def irreducible_character(lam, beta):
    result = 1
    for i in range(len(beta)):
        result *= murnaghan_nakayama(lam, [beta[i]], beta)
    return result

def specht_coefficient(f, lam, chi_lam):
    return sum(c_f * chi_lam for c_f, sigma in f.items() if len(sigma) == len(lam))

def effective_specht_support(f, chi_table):
    numerator = 0
    denominator = 0
    for lam, dim_vlam in chi_table.items():
        alpha_lam = specht_coefficient(f, lam, chi_table[lam])
        numerator += dim_vlam * alpha_lam ** 2
        denominator += dim_vlam * alpha_lam ** 4
    return (numerator / denominator) ** 0.5

def generate_random_formula(n, s):
    if n < 1 or s < 1:
        raise ValueError("n and s must be at least 1")

    def is_disjoint(a, b):
        return not any(x in b for x in a)

    def generate_node():
        row_set = random.sample(range(n), random.randint(1, n))
        col_set = random.sample(range(n), random.randint(1, n))
        bijection = {i: j for i, j in zip(row_set, col_set)}
        weight = random.random()
        return (row_set, col_set, bijection, weight)

    def generate_tree(depth):
        if depth == 0:
            return generate_node()
        else:
            left = generate_tree(depth - 1)
            right = generate_tree(depth - 1)
            while not is_disjoint(left[0], right[0]) or not is_disjoint(left[1], right[1]):
                left = generate_tree(depth - 1)
                right = generate_tree(depth - 1)
            return ("⊗", left, right)
```

```
    tree = generate_tree(s - 1)

    def evaluate_tree(node):
        if isinstance(node, tuple) and node[0] == "⊗":
            left = evaluate_tree(node[1])
            right = evaluate_tree(node[2])
            result = {}
            for sigma in left:
                for tau in right:
                    new_sigma = sigma + tau
                    result[new_sigma] = (left[sigma] * right[tau]) if is_disjoint(sigma, tau) else
            return result
        else:
            row_set, col_set, bijection, weight = node
            result = {}
            for sigma in itertools.permutations(range(n)):
                if all(bijection[i] == sigma[j] for i, j in enumerate(row_set)):
                    result[sigma] = weight
            return result

    f = evaluate_tree(tree)

    chi_table = {lam: irreducible_character(lam, beta) for lam in itertools.combinations(range(n),

    return f, chi_table

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    n_values = [4, 5, 6, 7]
    s_values = [1, 2, 4, 8, 16, 32]

    for n in n_values:
        for s in s_values:
            f, chi_table = generate_random_formula(n, s)
            supp_eff = effective_specht_support(f, chi_table)
            results.append({
                "n": n,
                "s": s,
                "supp_eff": supp_eff
            })

    mean_supp_eff = sum(result["supp_eff"] for result in results) / len(results)
    max_supp_eff = max(result["supp_eff"] for result in results)
    support_fraction = sum(1 for result in results if result["supp_eff"] <= result["s"]) / len(res

    conjecture_holds = support_fraction >= 0.8 and mean_supp_eff <= 1 and max_supp_eff <= 1.05
    counterexample = "" if conjecture_holds else f"max(supp_eff)={max_supp_eff} > s"

    return {
        "metric_name": "effective_specht_support",
        "metric_value": mean_supp_eff,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
```

```
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_supp_eff = sum(result["metric_value"] for result in results) / len(results)
    max_supp_eff = max(result["metric_value"] for result in results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_supp_eff} std=0.0 support_fraction={support_fraction}
    elif any(result["metric_value"] > 1.05 for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["metric_v
        print(f"RESULT: FALSIFIED counterexample='max(supp_eff) > s' first_failing_seed={first_fai
    else:
        print(f"RESULT: INCONCLUSIVE reason=not_enough_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ae01e1a.py", line 148, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ae01e1a.py", line 119, in run_trial
    f, chi_table = generate_random_formula(n, s)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ae01e1a.py", line 107, in generate_ran
    chi_table = {lam: irreducible_character(lam, beta) for lam in itertools.combinations(range(n), n /
                                                ^^^^
NameError: name 'beta' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a NameError ('beta' is not defined) before producing any data, so the pre-registered support condition cannot be evaluated. | next: Fix the bug in generate_random_formula (define/pass 'beta' to irreducible_character) and rerun the full 720-trial sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	220623
2	preregistration	claude_max	opus	0	0	7967
3	novelty	claude_max	opus	0	0	3944
4	novelty	claude_max	opus	0	0	7274
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14794
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16100
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15672
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17358
9	judge	claude_max	opus	0	0	3877

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 307608 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/b5cefcac0023.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b5cefcac0023.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b5cefcac0023.tar.gz` (if generated)